ROS 2 vs. classic Python 3 for RockyBorg

Purpose: provide a project-specific, evidence-based comparison to support a team decision. This document compares two architectures; it does not record a final choice.

Reviewed: Ismael Ismael — 2026-10-29
Project context: current goals and meeting notes, hardware order list, and the existing `RockyBorg_Python_vs_ROS2.docx` comparison.

Short answer

For the first demonstrator—reading the proposed sensors, detecting a line, and controlling a motor—classic Python 3 is likely the lower-risk route for a small beginner team on a short schedule. ROS 2 becomes more valuable if the assessed outcome depends on multiple independently developed robot components, standard message-based communication, visualization/simulation workflows, or a credible path to sensor fusion and SLAM.

That is a provisional assessment, not a decision. Both options use Python for much of the application code, and both can run on Linux. The real choice is **a lightweight Python application versus a ROS 2 robotics framework**, including its packages, communication model, tooling, installation and runtime.

What the project appears to require

The project notes describe a staged autonomous robot:

1. Bring up hardware and verify motor, camera, sensors, and communication.
2. Follow a straight line, then a curved path, and collect telemetry.
3. Stream and combine sensor data; consider object detection.
4. Investigate localization, SLAM, and 3D mapping.

The proposed hardware list names a Raspberry Pi 4 Model B with 8 GB RAM, Raspberry Pi Camera Module 3, BME280, a digital light-sensor module, and a laser-distance module. It does not list a motor or motor driver, and the notes say their availability is unknown. Stock status for some listed parts is uncertain. Therefore, motor control and sensor compatibility cannot yet decide the software choice.

The existing comparison document describes a three-person team with limited programming experience and limited development time. Treat those as planning assumptions to confirm with the team.

The options, stated precisely

Classic Python 3

Write one application, or a small set of Python modules/processes, and connect the hardware with suitable Linux/Python libraries. The team chooses how components pass data (function calls, queues, sockets, MQTT, etc.) and builds or selects any required logging, configuration, process supervision, and message formats.

ROS 2 with Python nodes

Write Python nodes using `rclpy`, ROS 2's Python client library. Nodes exchange typed messages through topics and use services or actions for request/response and longer-running work. ROS 2 supplies conventions and tools for composing, launching, inspecting, and connecting robot components. The nodes may still call Python libraries such as OpenCV and camera/sensor APIs.

ROS 2 is not a different programming language from Python, and it is not limited to Linux. ROS 2 supports more than one operating system, but support and installation differ by ROS distribution. The current ROS 2 documentation provides Windows 11 installation instructions for its current development distribution; a deployment on the Pi still needs a compatible Linux image and ROS distribution. Do not assume that instructions for one ROS distribution apply unchanged to another.

Project-specific comparison

Decision factor	Classic Python 3	ROS 2 with Python nodes	What it means for RockyBorg
First hardware test	Fewer framework concepts; call the chosen device libraries directly.	Must install and configure ROS, create a workspace/package, and understand nodes and interfaces as well as the device library.	Python is likely faster for the first GPIO/I2C/camera/motor smoke checks, provided the libraries support the chosen Pi OS.
Straight-line following	A single loop can read a light sensor or process camera frames and set motor commands.	Separate camera/sensor and motor/controller nodes can communicate through topics.	One or two tightly coupled components do not, by themselves, require ROS 2. A simple milestone can be demonstrated with either.
Camera Module 3	Raspberry Pi documents Picamera2 as a Python camera interface on Raspberry Pi OS.	A ROS node can use Picamera2/OpenCV and publish images, or use a ROS camera driver if a compatible one is available.	Picamera2 is a practical route for the listed camera on Raspberry Pi OS. Confirm the exact OS and any ROS camera-driver integration before committing.
BME280/light/distance sensors	Direct libraries or device interfaces; each part needs a small driver/adaptor and tests.	The same hardware libraries can be wrapped in ROS nodes and publish standard or project-defined messages.	Neither framework makes an electrically or API-incompatible breakout work. Verify each exact board, bus, voltage, and Python package first.
Motor control	A dedicated motor-driver library or GPIO interface can be called directly; the	A hardware node can expose commands to a controller; `ros2_control` may help for supported controllers and	The motor and driver are not yet confirmed, so test their actual interface before estimating effort. Do not drive a motor directly from Pi GPIO.

Decision factor	Classic Python 3	ROS 2 with Python nodes	What it means for RockyBorg
	motor driver remains required.	hardware interfaces, but it is additional integration work.	
Curved-path controller and telemetry	Straightforward to add in one application; logging format and boundaries are the team's responsibility.	Controller, telemetry, and sensor nodes can be separated; ROS topics and tools provide clearer runtime data flow.	ROS 2 helps when the number of independently changing components grows. For a small prototype, a well-structured Python package can be enough.
Sensor fusion/localization/SLAM	Possible, but the team must select, integrate, and connect libraries and data formats itself.	Stronger robotics ecosystem and standard interfaces make integrating ROS-compatible navigation, mapping, and sensor packages more plausible.	This is ROS 2's strongest argument if SLAM or navigation is a required, assessed deliverable. A future idea alone may not justify the setup cost now.
Simulation	Can connect to a simulator, but the bridge and message plumbing are team responsibilities.	ROS 2 has established integration paths with modern Gazebo and ROS messages. Simulator, ROS distribution, and plugin versions must match.	Useful if Gazebo simulation is a required milestone; otherwise, simulation may add setup work before hardware basics are proven.
Development on Windows 11	Python and editor workflow are direct for logic that does not depend on Pi-only hardware libraries. Hardware code still needs a Linux/Pi test environment.	ROS 2 Windows support exists for supported distributions, but is not identical to a Raspberry Pi Linux deployment. WSL/VM/container setup can introduce networking, device access, or GUI friction.	Avoid choosing ROS 2 solely on an assumption that it cannot run on Windows. Instead test the exact distro and plan a Linux target environment that matches the robot.
Raspberry Pi deployment	Raspberry Pi OS has first-party camera tooling and a Python Picamera2 API. Package and	ROS 2 can target ARM64 Linux, but the Pi OS/Ubuntu image, ROS distribution, Python version, and device packages must align. A	The OS choice is coupled to both the camera stack and ROS availability. A Pi 4 with 8 GB makes ROS 2 plausible, but does not remove compatibility or setup work.

Decision factor	Classic Python 3	ROS 2 with Python nodes	What it means for RockyBorg
	wheel compatibility still needs checking for OpenCV and sensor libraries.	development build may be needed on unsupported combinations.	
Resource use and timing	Fewer framework processes and layers can reduce overhead, but image processing and poorly designed loops can still overload the Pi.	Adds middleware and node/runtime overhead, but enables modular deployment and can distribute computation across Pi and computer.	There is no reliable project-specific CPU/RAM winner without measuring the chosen camera resolution, frame rate, message transport, and algorithm. Benchmark both with the same workload if resource use matters.
Team learning and debugging	Smaller initial concept set; risk of creating ad-hoc interfaces and one large script if boundaries are not maintained.	More concepts and setup up front; clearer interfaces and runtime tools once the team knows nodes, topics, packages, and launch.	The team's available learning time and whether multiple people will work on separate components are decisive.

Important corrections to the earlier comparison

- "ROS 2 versus Python" is an imprecise comparison. ROS 2 Python nodes are Python programs; the decision is whether to use the ROS 2 framework.
- ROS 2 is not inherently Linux-only. Current official docs include Windows 11 support for the current distribution. The Pi's deployment OS and compatibility with a selected ROS distribution still matter.
- "Python uses less CPU/RAM" and "ROS 2 uses more CPU/RAM" should not be treated as established for this robot. Measure the same camera/control workload on the Pi; image resolution, frame rate, encoding, transport, and algorithms may dominate.
- ROS 2 is not a prerequisite for a basic line follower or MQTT/Node-RED data exchange. It is most compelling when the team needs ROS's component communication and robotics ecosystem.
- Choosing classic Python now does not make later ROS 2 impossible. Keep hardware access, line detection, controller logic, and telemetry behind small interfaces so they can be wrapped by ROS 2 nodes later. That preserves some logic, though migration still requires integration and testing.

Main risks by option

If the team chooses classic Python 3

- The team must define stable interfaces, timestamps, units, configuration, logging, and failure behavior itself.

- A monolithic control script can become difficult to share across three people; split code into small modules and testable hardware adapters early.
- Integrating future SLAM or ROS-oriented packages may require bridges or a later migration.
- Pi OS camera support is convenient, but OpenCV wheels can lag specific ARM/Python/OS combinations. Test installation on the actual target rather than assuming the Windows development environment proves deployment.

If the team chooses ROS 2

- Initial setup, package/workspace conventions, node design, messages, launch, and debugging consume project time before the robot moves.
- The team must pick compatible ROS distribution, OS, architecture, Python version, camera stack, and sensor packages; test these together early.
- Sending full camera frames over the network can cost bandwidth and CPU. Prefer local vision on the computer/Pi decision based on measurement; publish reduced-rate or compressed data if appropriate.
- A framework does not supply a driver for every exact sensor or motor controller. Custom integration may still be required.
- Advanced ROS capabilities only pay off if the project reaches the milestones that need them.

A fair decision test before committing

Use the same first milestone as a small proof of feasibility in each approach. Time-box each attempt and record actual results rather than estimating from framework reputation.

1. On the target Raspberry Pi and chosen OS, capture a camera frame and read one available sensor.
2. Read a line sensor or process a low-resolution camera frame and produce a steering value.
3. Send a safe command through the actual motor driver once identified; record loop rate, CPU/RAM, setup time, and failure/recovery behavior.
4. For ROS 2, split sensor and controller into nodes and pass the value through a topic. For classic Python, use the simplest clean module/process interface the team would keep.
5. Have a different team member install and run the result using written setup instructions.

Choose ROS 2 if its modularity/tools are needed for planned deliverables and the setup test fits the schedule. Choose classic Python if the graded target is basic control and sensing, and ROS 2 setup would displace implementation time. If the course requires SLAM, multi-sensor integration, or ROS specifically, that requirement outweighs the short-term convenience argument and should be confirmed with the instructor.

Provisional conclusion

Based on the currently documented scope and constraints, **classic Python 3 is the safer starting hypothesis for milestones 1–3**, while **ROS 2 is the stronger choice if ROS-based simulation, multi-node integration, navigation, or SLAM is a real project requirement**. The Raspberry Pi 4's 8 GB does not alone rule ROS 2 in or out. The decision should wait until the team confirms required deliverables, OS, motor/driver, and a short on-device feasibility test.

Sources

Project-specific sources:

- Project meeting notes
- Hardware order group 4
- Current Python vs. ROS 2 comparison

Official technical documentation:

- ROS 2 Jazzy release and platform/support information
- ROS 2 current installation options
- ROS 2 Jazzy topics concept
- ROS 2 Jazzy `rclpy` API
- ROS 2 Windows 11 binary installation documentation
- Raspberry Pi camera software and Picamera2 documentation
- OpenCV Python installation and Raspberry Pi/ARM notes